

### \* Decorators in python :-

- Decorator is one of the function which will provides Additional processing capability to the normal function value and returns the modified value.
- A Decorator function is always takes Normal function as parameter.

#### syntax :

```
def functionname1(functionname): # Decorator
    def innerfunctionname(): # inner function name
        val = functionname()
        # do the operation on 'val'
        return result # inner function must return modified value
    return innerfunctionname # decorator returns inner function name
```

⇒ Here functionname1 is called "Decorator function"

⇒ Here functionname as a formal parameter. Every decorator function must take normal function as parameter.

# program for demonstrating non-decorator example.

```
def getval():
    return int(input("Enter any value:"))
```

```
def square():
    n = getval()
    res = n**2
    return res.
```

```
def squareroot():
    n = getval()
    res = n**0.5
    return res.
```

```
def cube():
    n = getval()
    res = n**3
    return res
```

# main program

```
res = square()
print("square = {}".format(res))
print("-----")
res = squareroot()
print("square root = {}".format(res))
print("-----")
res = cube
print("cube = {}".format(res))
```

# program for demonstrating ~~name~~ decorator

ex ① . def getval(): # This function defined by kur  
return int(input("Enter Any value:"))

def square(kur): # here square is called Decorator  
def operation(): # inner function with the  
operations for kur function  
(alias name of getval())  
n = kur()  
res = n \*\* 2  
return res  
return operation

# main program

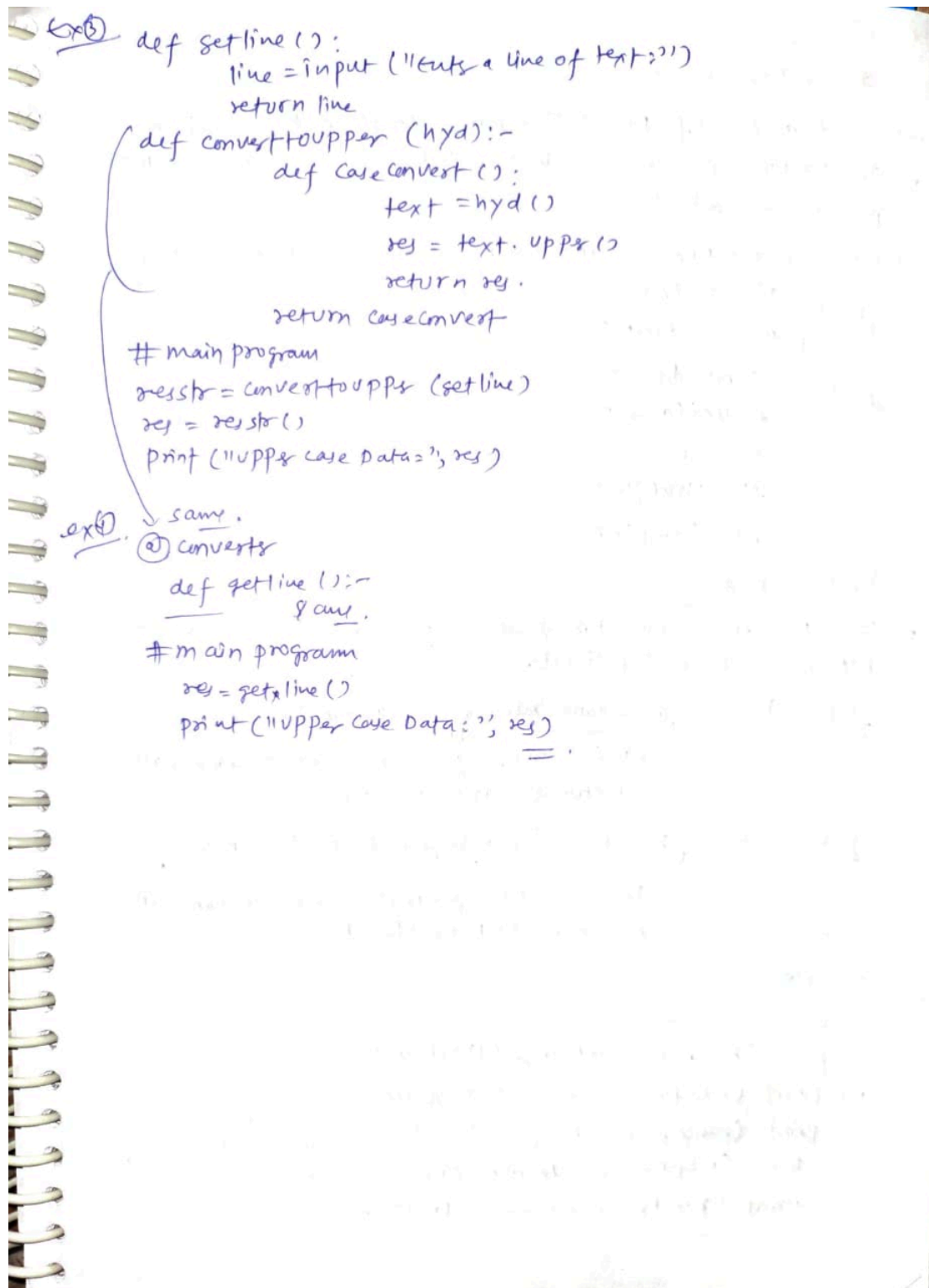
```
result = square(getval)
res = result()
print("square = {}".format(res))
```

ex ② same .  
a) square

```
def getval(): # this function is defined by kur
return int(input("Enter any value:"))
```

# main program

```
res = getval() # calling Normal function
print("square = {}".format(res))
```



```
In [1]: #program for demonstrating non-decorator
def getval(): # This Function Defined by arig
    return int(input("Enter Any Value:"))

def square():
    n=getval()
    res=n**2
    return res

def squareroot():
    n=getval()
    res=n**0.5
    return res
```

```

def cube():
    n=getval()
    res=n**3
    return res

#Main program
res=square()
print("Square ={}".format(res))
print("-----")
res=squareroot()
print("Square Root ={}".format(res))
print("-----")
res=cube()
print("Cube ={}".format(res))

```

Square =7569

Square Root =7.3484692283495345

Cube =9261

In [4]:

```

####Program for demoinstrating decorator
#DecEx1
def getval(): # This Function Defined by arif
    return int(input("Enter Any Value:"))

def square( kvr ): # Here Square is called Decorator
    def operation(): # Inner Function will the operations for kvr function
        n=kvr()
        res=n**2
        return res
    return operation

#main program
result=square(getval)
res=result()
print("Square={}".format(res))

```

Square=625

In [5]:

```

#Ex2
def square(kvr):
    def operation():
        n=kvr()
        res=n**2
        return res
    return operation

@square
def getval(): # This Function Defined by KVR
    return int(input("Enter Any Value:"))

#main program
res=getval() # Calling Normal Function
print("Square={}".format(res))

```

Square=625

```
In [7]: #Ex3.py
def getline():
    line=input("Enter a Line of Text:")
    return line

def converttoupper(hyd): # here converttoupper() is called Decorator
    def caseconvert():
        text=hyd()
        res=text.upper()
        return res
    return caseconvert

#Main Program
resstr=converttoupper(getline)
res=resstr()
print("Upper Case Data=",res)
```

Upper Case Data= MY NAME IS ARIF

```
In [8]: #DecEx4
def converter(kvr):
    def caseconvert():
        text=kvr()
        res=text.upper()
        return res
    return caseconvert

@converter
def getline():
    line=input("Enter a Line of Text:")
    return line

#main program
res=getline()
print("Upper Case Data:",res)
```

Upper Case Data: IM ARIF RAZA

In [ ]: